Introduction to R

Transforming Publishing
phs.transformingpublishing@phs.scot

Public Health Scotland

Pathway

Intro
Intro to Data and Tools, Overview of R

Foundations
Commenting, Types, Variables, Statements, Data Structures, Packages

Posit Workbench
Desktop/Server Version, Interface, Customisation, R Scripts, Hints/Tips

Workflow
Overview (Collect, Explore, Wrangle, Viz, Outputs), Git(Hub/ea), RMarkdown, RAP, Templates, Style Guide

Data Flow
Directories/File Paths, CSVs, SPSS (haven), SMRA/Other Databases

Explore
Mean, Median, Summary Function, Frequencies/Cross-Tabs

Wrangle
Tidyverse (dplyr/magrittr), Pipes, Functions (Filter, Mutate, Arrange, etc.), PHS Methods

Visualise
Intro to ggplot2, Line Graphs, Bar Plots, Scatterplots, Customisation

Output
Overview of RMarkdown, Shiny, etc.

Review
Overview, Next Steps, Q&A



Introduction – the why

R

- is a **programming language** widely used for data analysis, statistics, and graphics;
- **is open source**, available on all major operating systems;
- has the functionality to go from raw data to interactive reports and web apps;
- and it's part of the **PHS analytical strategy.**



Foundations – the what

Building Blocks

- **Basic Data Types** – how is fundamental data, like numbers and text, represented?

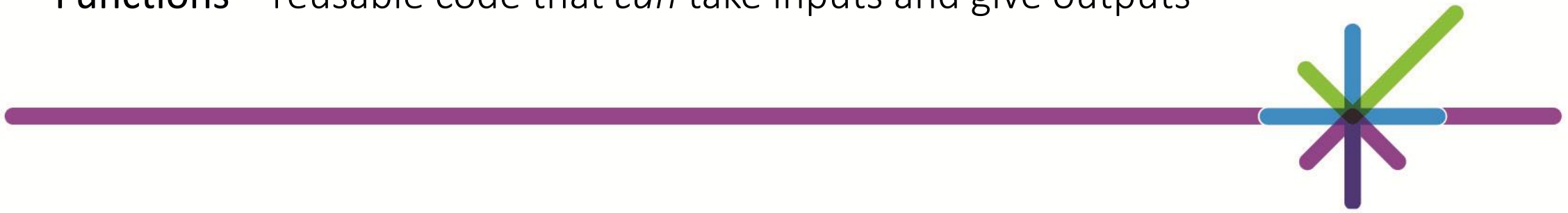
  This is then the foundations of more complex, composite data types, e.g. tables.

- **Variables** – named storage to track "objects" across our program

- **Statements** – a complete line of code, made of expressions and operators

- **Control Flow** – branching (if statements) and iteration (loops)

- **Functions** – reusable code that *can* take inputs and give outputs



Basic Data Types

- **Character** (String) – e.g. `"Hello World!"`
- **Numeric** (Float/Real) – e.g. `123.5`
- **Logical** (Boolean) – e.g. `TRUE`

```
typeof("Hello World")
is.numeric(123.5)
print(typeof(TRUE))
```

```
> [1] "character"
> [1] TRUE
> [1] "logical"
```



Type Conversion

`as.<data_type>()`

- `as.character()` conversions tend to succeed without fault
- `as.numeric()` – TRUE and FALSE become 1 and 0, character types needs to be formatted correctly
- `as.logical()` – everything except 0 becomes TRUE for numeric conversions, character can be upper, lower, or proper case versions

```
as.character(123.5)
as.numeric("123.5")
as.logical("False")
```

```
> [1] "123.5"
> [1] 123.5
> [1] FALSE
```



Variables

- **Way of saving data to the memory to be used later in a program** – can save single numbers or entire datasets
- **Assignment** – variables are assigned mainly with `<-` *you can use the keyboard shortcut alt+-*
- **Naming** – Must be all one word. Letters, numbers, dots `.` or underscores `_` are all okay. However, you **can't** start with an underscore or number, or a dot then a number. Any existing terminology is also reserved from being used as a variable. *Following style guidance is also important.*

```
# Will Work
Name <- "Paige"
sumOfPatientsUnder60 <- 600

## Will not work
_name <- "Ali"
60YearOldPatients <- 150
sum of patients under 60 <- 600
TRUE <- "2 + 2 = 4"
.5 <- "half"
```



Mathematical Operators

Precedence	Operator	Description
1	^	To the power of/ Exponentiation
2	%%	Modulus
3	* /	Multiplication, Division
4	+ −	Addition, Subtraction
5	< > <= >= == !=	Comparison Operators (Less Than, More Than, Less Than or Equal To, More Than or Equal To, Equal To, Not Equal To)
6	!	Logical NOT
7	& &&	Logical AND
8	| ||	Logical OR



Knowledge Check

We’re going to use an app for a lot of our interactive work, especially today. (Use hint button if you get stuck).

scotland.shinyapps.io/phs-rtraining-intro/

- Foundations



Anatomy of a Program

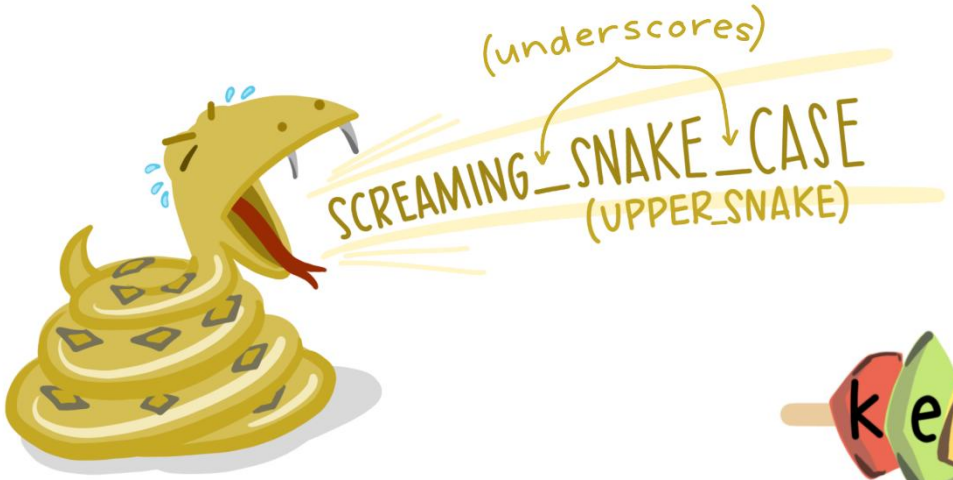
```
# Example 1
hello_world <- "Hello World"
print(hello_world)
```

```
> [1] "Hello World"
```

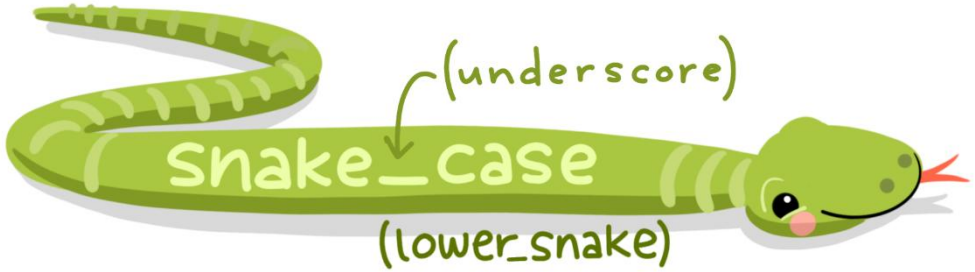
- `# Example 1` – comment
- `hello_world` – variable
- `<-` – assignment operator (`alt + -`)
- `"Hello World"` – character (" or ')
- print() – function



Coding Case Types











Style Guide

- **Naming** – variables and filenames should have meaningful names in snake_case format, preferring all lower case.

- **Comments** – use # to write comments that section out the code and explain what it does

- **Structure**
  - Space after a comma
  - No spaces before or after parenthesisPrefer `"` over `'` for characters

- More Info: [PHS Data Science - Knowledge Base (public-health-scotland.github.io)]

```
# Bad
ptsNms <-c ( 'Ali','Paige','Mo')

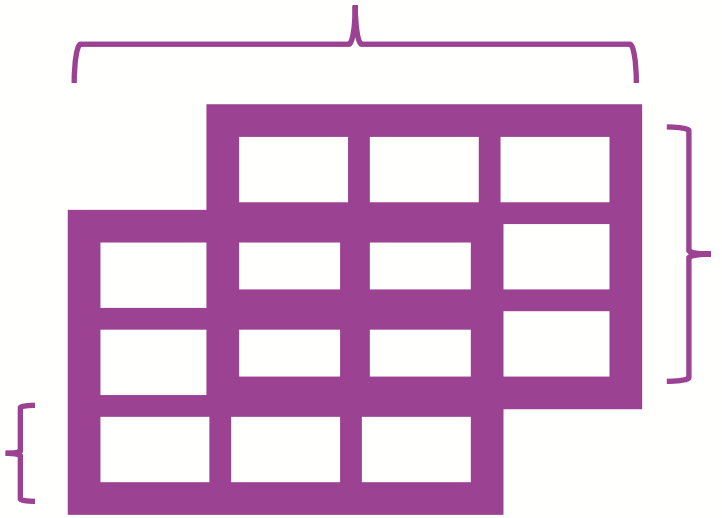
# Good
# Assign Patient names:
patient_names <- c("Ali", "Paige", "Mo")
```



Foundations – the what

Data Structures

- Vectors
- Lists
- Factors
- Matrices
- Data Frame



- `str()` provides an overview and description of the data structure



Vectors

```r
# create vector and assign
patient_names <- c("Ali", "Paige", "Mo", "Ailsa", "Joseph")
numbers <- 1:5
vector("character", 5)
vector("logical", 5)
```

contain multiple objects of the *same* basic class

- Create: `c(...)` or `vector(<type>, <length>)`
- Access: `<vector>[<index>]` index can be a number or a name if the vector has assigned them using `names()`

```
> # call specific entry in vector:
> c("Ali", "Paige", "Mo", "Ailsa", "Joseph")[2]
[1] "Paige"
> patient_names[2]
[1] "Paige"
> numbers[2]
[1] 2
> # Call range of entries in vector:
> c("Ali", "Paige", "Mo", "Ailsa", "Joseph")[2:4]
[1] "Paige" "Mo"    "Ailsa"
> patient_names[2:4]
[1] "Paige" "Mo"    "Ailsa"
> numbers[2:5]
[1] 2 3 4 5
```



Lists

are a special type of vector that can contain objects of *different* classes, including other lists

- Create: `list(...)`
- Sub-list: `<list>[<index>]`
- Access: `<list>[[<index>]]`

```
list("abc", 4, FALSE)[1:2]
list(list(2, 3), "abc")[[2]]
```

```
> [[1]]
> [1] "abc"
> [[2]]
> [1] 4
```

```
> [1] "abc"
```



Naming Elements

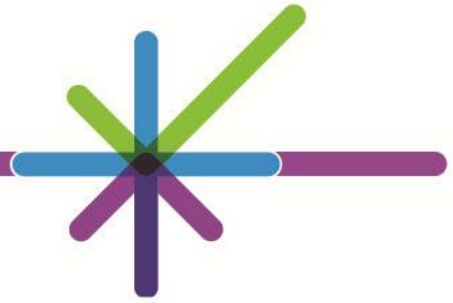
can be done on vectors and lists during or after creation.

- At creation:
  `c("<name>", = <item>)` or
  `list("<name>", = <item>)`

- After:
  `names(<object>) <-`
  `c("<name>")`

```
x <- list("Ch" = "a", "Nm" = 2)
names(x) <- c("Char", "Num")
x$Char
```

```
> [1] "a"
```



Knowledge Check

scotland.shinyapps.io/phs-rtraining-intro/

- **Data Structures –** 1st question “Vectors”



Factors

are used to represent categorical data, with both ordered and unordered variations.

- Create: `factor(c(...), ordered = <TRUE/FALSE>)`
- Levels: `factor(c(...), levels = c(...))`

```
> ## create and assign factors
> x <- factor(c("M", "F", "M", "F", "M"),
+             levels = c("F", "M"))
> x
[1] M F M F M
Levels: F M
> age <- factor(c("elderly", "child", "child", "elderly", "adult"),
+               levels = c("child", "adult", "elderly"),
+               ordered = TRUE)
> age
[1] elderly child   child   elderly adult
Levels: child < adult < elderly
```



Knowledge Check

scotland.shinyapps.io/phs-rtraining-intro/

- **Data Structures –** Question 2 Factors



Matrices

expand our dimensions with a `nrow` and `ncol` arguments, constructed column-wise

- Create: `matrix(<data>, nrow = <int>, ncol = <int>)`
- Access: `<matrix>[<row>, <col>]`

```
x <- matrix(1:6, 2, 3)
x
x[2, 3]
```

```
>      [,1][,2][,3]
> [1,]    1   3   5
> [2,]    2   4   6
```

```
> [1] 6
```



Data Frames

are used to store tabular data, each column contains one variable, each row contains an observation

- Create: `data.frame("<name>" = <element(s)>)`
- Subset: `[]`
- Access: `[[]]` or `$`

```
> ## data frame
> data_frame <- data.frame(
+   patient_names = c("Ali", "Paige"),
+   sex = factor(c("M", "F"),
+                levels = c("F", "M")),
+   age_group = factor(c("elderly", "child"),
+                 levels = c("child", "adult", "elderly"),
+                 ordered = TRUE),
+   age = c(83, 12))
> data_frame
  patient_names sex age_group age
1           Ali   M   elderly  83
2         Paige   F     child  12
> data_frame[4]
  age
1  83
2  12
> data_frame["age"]
  age
1  83
2  12
> data_frame$age
[1] 83 12
> data_frame$age[2]
[1] 12
> data_frame[2,4]
[1] 12
```

Tibbles

Tibbles are data frames that attempt to make our lives a bit easier

First, load the package:
`library(tibble)`

- **Create**: `tibble("<name>" = <element(s)>)` or coerce an existing data frame with `as_tibble(<object>)`

- **Subset and Access**: `[]` `[[]]` or `$`

```
tibble(name = c("Harry",
                "Sarah"),
       score = c(62, 91))
```

```
>      name  score
>  1  Harry     62
>  2  Sarah     91
```



Knowledge Check

scotland.shinyapps.io/phs-rtraining-intro/

- Data Structures – Question 4 Data Frames



Foundations – the what

Anatomy of a Function

Functions allow us to bundle code for reuse, taking inputs, doing something to them, then providing outputs.

```
<name> <- function(<inputs…>){
    <code>
    return(<outputs…>)
}
```

```
mult_2 <- function(x){
    x <- x * 2
    return(x)
}
```

```
mult_2(4)
```

```
> [1] 8
```



Packages

are used to expand the functionality of R with pre-built functions

- Install: `install.packages("<package>")`

- Load: `library(<package>)` or `require(<package>)` if loading packages as part of functions as it returns a logical value and a warning if the package isn't installed.

```
install.packages("tidyverse")
library(tidyverse)
```

  - *output varies by package, warnings (not errors) are normal. An example would be where a function 'masks' that of another R function.*

- Help with loading packages: PHS Data Science - Knowledge Base (public-health-scotland.github.io)



Control Flow - if statements

Package: `dplyr`

Load package, `library(dplyr)`

- `if_else(<condition>, <if true do this>, <if false do this>)`

```
library(dplyr)
x <- 5
if_else(x > 10, TRUE, FALSE)
```

```
> [1] FALSE
```



Knowledge Check

scotland.shinyapps.io/phs-rtraining-intro/

- Control Flow – Question 1 If



Control Flow - case statements

Package: `dplyr`

Load package, `library(dplyr)`

- `case_when(<condition> ~ <result>)`
- To set everything else at the end use the condition `TRUE ~ <result>`

```
library(dplyr)
x <- c(1, 2, 3, 4, 5)
case_when(x < 3 ~ "LT3",
          x %% 2 == 0 ~ "Even")
```

```
> [1] "LT3" "LT3" NA "Even" NA
```

```
data %>% case_when(
  is.na(col_name) ~ 0,
  TRUE ~ col_name
)
```



Control Flow - case statements

dplyr::case_when() IF ELSE... (but you love it?)

```
df %>% ADD COLUMN 'danger'
  mutate(danger = case_when(type == "kraken" ~ "extreme!",
                            TRUE ~ "high"))
```

IF type is kraken THEN danger is extreme!

OTHERWISE, danger is high.



Knowledge Check

scotland.shinyapps.io/phs-rtraining-intro/

- Control Flow – Question 2 Case



Iteration – for loop

allowing us to do the same thing repeatedly with different inputs.

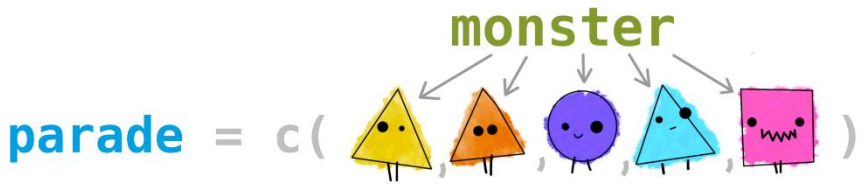
- `for(<value> in <sequence>) {<statement>}`

```
> for (month in 1:5) {
+   print(paste('Month:', month))
+ }
[1] "Month: 1"
[1] "Month: 2"
[1] "Month: 3"
[1] "Month: 4"
[1] "Month: 5"
```

```
files <- list.files(pattern = ".csv")
all_files <- list()
for(i in seq_along(files)) {
    all_files[[i]] <-
                read.csv(files[i])
}
```



Iteration – for loop



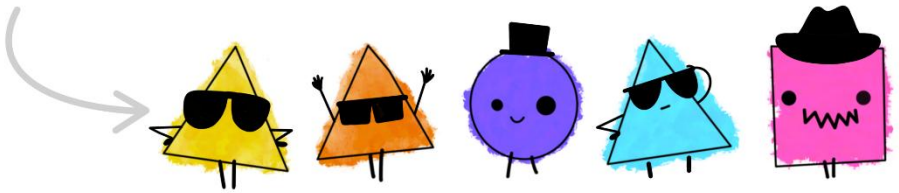

```
for (monster in parade) {

        if (shape(monster) == triangle) {
            monster_style = monster + sunglasses
        }

        else {
            monster_style = monster + hat
        }

        print(monster_style)

}
```



Iteration – loop with purrr

Package: `purrr`

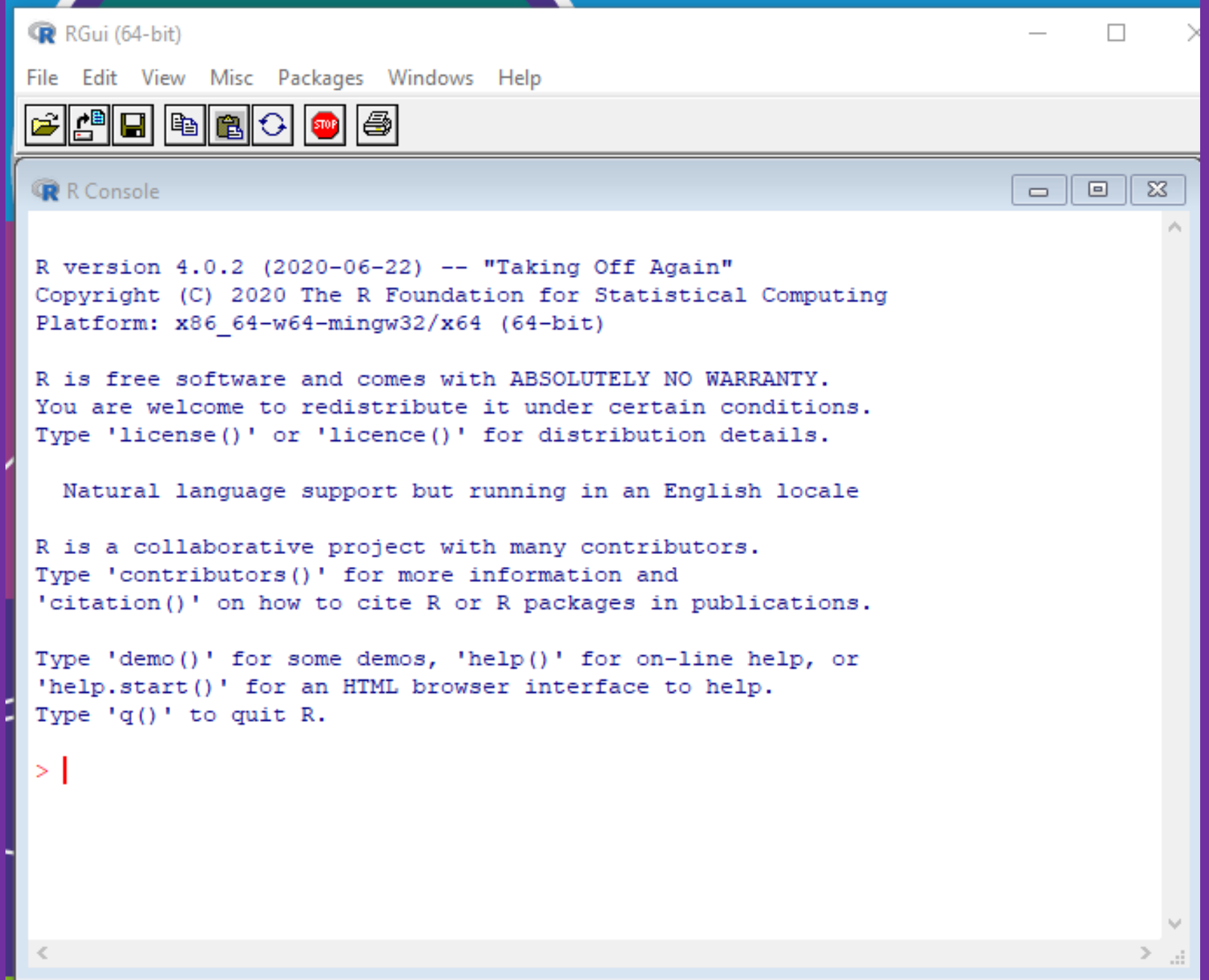
Load: `library(purrr)`

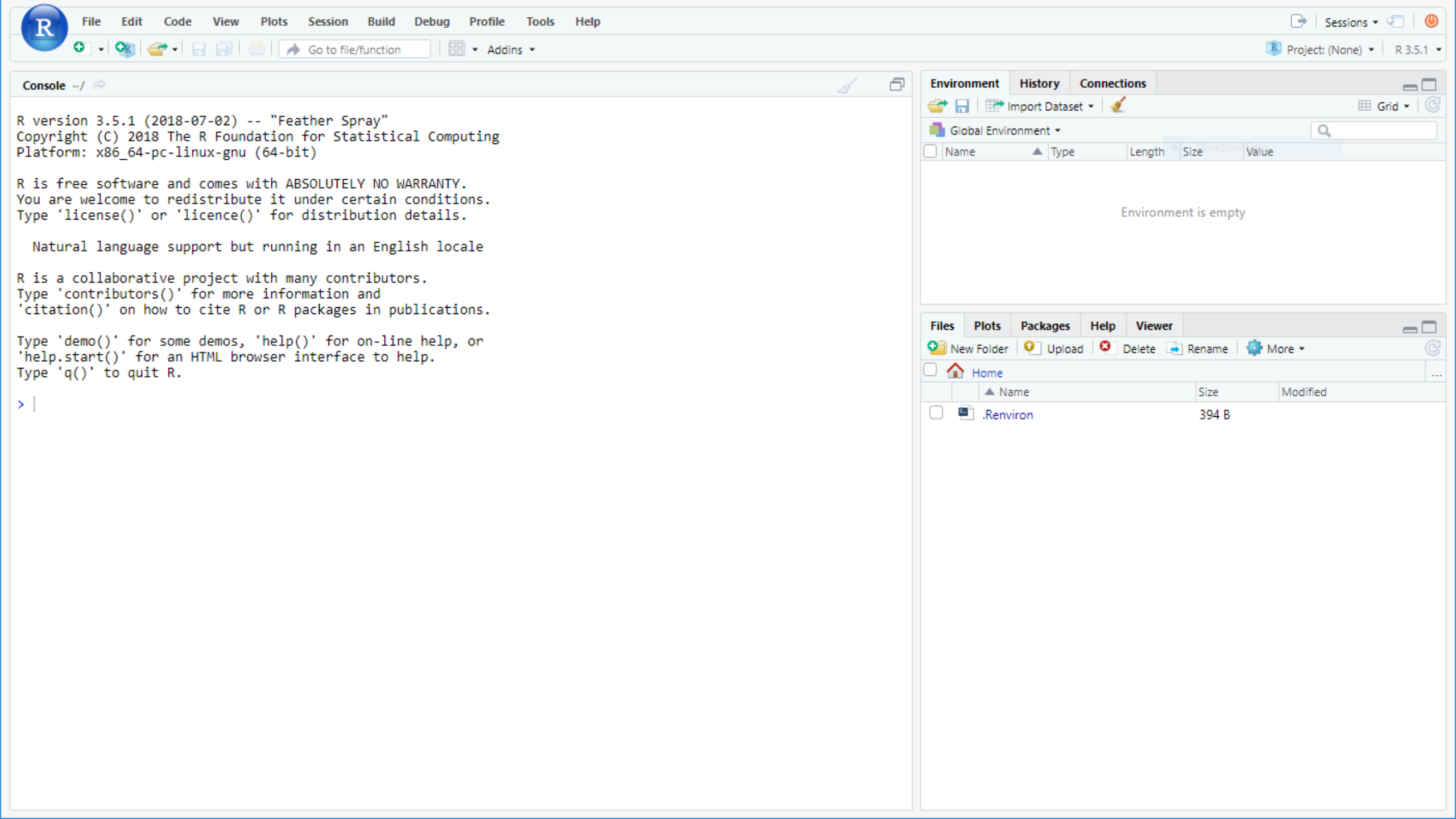
- `map(<object>, <function>)`

```
library(purrr)
files <- list.files(pattern = ".csv")
all_files <- map(files, read_csv)
```



Rstudio and Posit Workbench


RGui (64-bit)
File Edit View Misc Packages Windows Help
R Console
R version 4.0.2 (2020-06-22) -- "Taking Off Again"
Copyright (C) 2020 The R Foundation for Statistical Computing
Platform: x86_64-w64-mingw32/x64 (64-bit)
R is free software and comes with ABSOLUTELY NO WARRANTY.
You are welcome to redistribute it under certain conditions.
Type 'license()' or 'licence()' for distribution details.
Natural language support but running in an English locale
R is a collaborative project with many contributors.
Type 'contributors()' for more information and
'citation()' on how to cite R or R packages in publications.
Type 'demo()' for some demos, 'help()' for on-line help, or
'help.start()' for an HTML browser interface to help.
Type 'q()' to quit R.
>


File Edit Code View Plots Session Build Debug Profile Tools Help
Sessions
Go to file/function
Addins
Project: (None)
R 3.5.1
Console ~/
R version 3.5.1 (2018-07-02) -- "Feather Spray"
Copyright (C) 2018 The R Foundation for Statistical Computing
Platform: x86_64-pc-linux-gnu (64-bit)
R is free software and comes with ABSOLUTELY NO WARRANTY.
You are welcome to redistribute it under certain conditions.
Type 'license()' or 'licence()' for distribution details.
Natural language support but running in an English locale
R is a collaborative project with many contributors.
Type 'contributors()' for more information and
'citation()' on how to cite R or R packages in publications.
Type 'demo()' for some demos, 'help()' for on-line help, or
'help.start()' for an HTML browser interface to help.
Type 'q()' to quit R.
>
Environment History Connections
Import Dataset
Grid
Global Environment
Name Type Length Size Value
Environment is empty
Files Plots Packages Help Viewer
New Folder Upload Delete Rename More
Home
Name Size Modified
.Renviron 394 B

Posit Workbench

We don’t generally work on desktop, we work remotely using Posit Workbench:

RStudio Sign In (publichealthscotland.org)

- **Faster** – More computing power than your laptop when you need it
- **More Secure** – Can be used to analyse confidential datasets
- **More info** - PHS Data Science - Knowledge Base (public-health-scotland.github.io)
- **Quick Start Guide** – PHS Data Science - Knowledge Base (public-health-scotland.github.io)



Posit Workbench Guidance

- **Close Sessions** – leaving sessions open or running multiple sessions uses resource - when finished, close it down using the red power button in top right



- **Store Data Properly** – projects and files shouldn't be stored in your 'home' drive and unused data should be removed from your environment, use `rm(list = ls())` to clear anything

- **Retrieve Data Efficiently** – limit the data extracted using well written SQL

- **Further Guidance** – PHS Data Science - Knowledge Base (public-health-scotland.github.io)

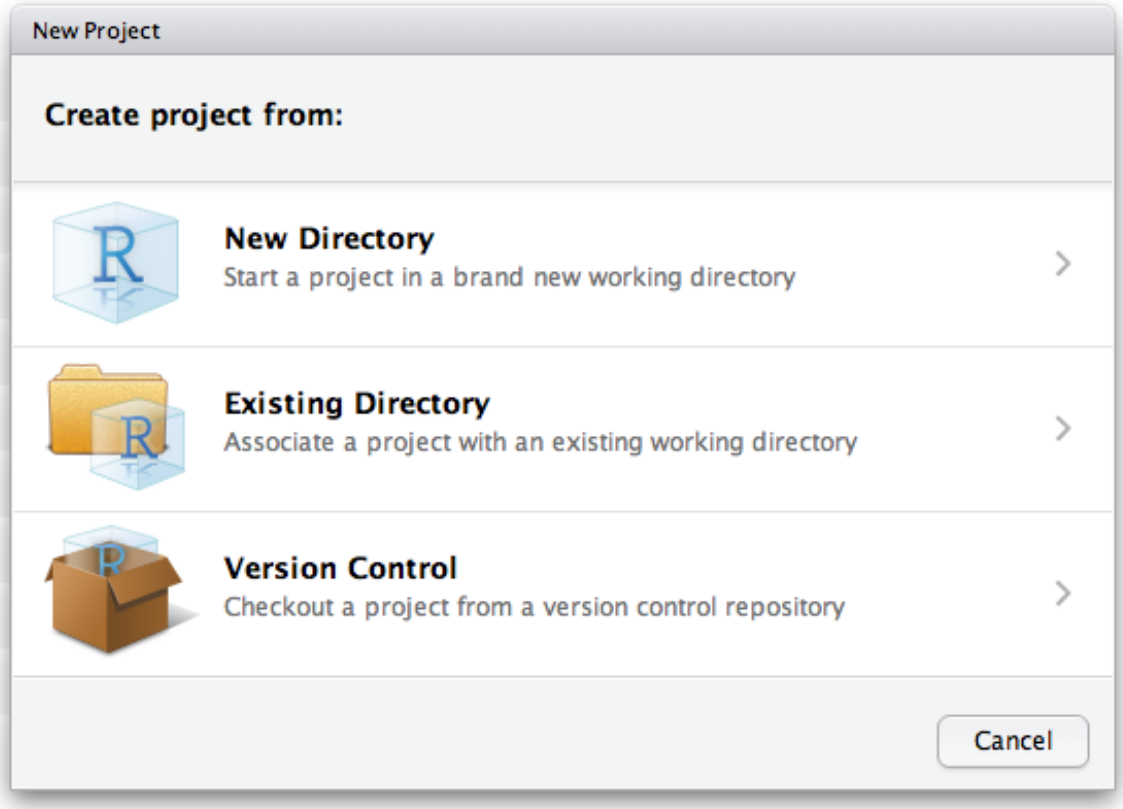
R Projects

keeps work separate, giving a project its own working directory, workspace, and history.

Opening an .Rproj file will:

- Start a new R session
- Load project specifics and settings
- The project directory is set as the current working directory.

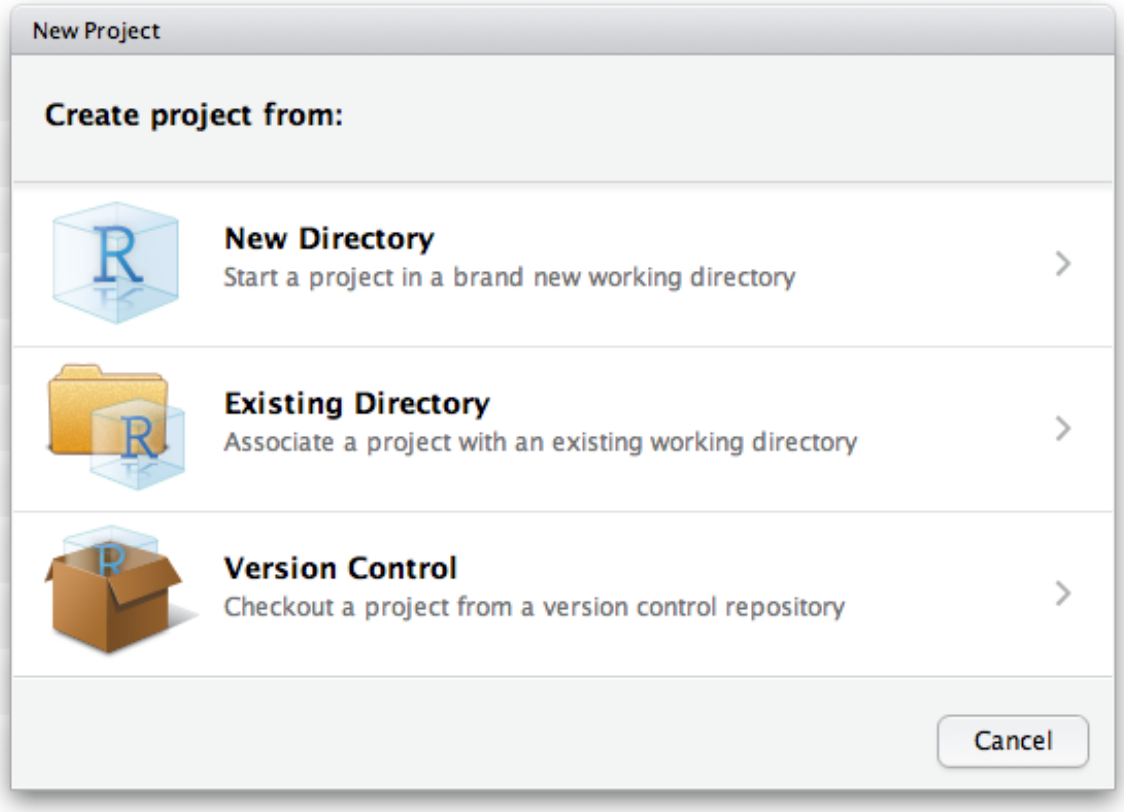
- **Create:** available in the Projects menu or global toolbar.



R Projects

Guidance for working with projects in Posit:

- What is a project (in Posit Workbench)?
- How do I open or switch to another project?



Data Flow

Working Directory

- Current: `getwd()`
- Set new: `setwd(<filepath>)`
- We can also use the `here` package, with `here()`
- File paths use forward slashes /

Rstudio also provides options through the user interface for navigating files and directories.

File paths on server are slightly different than they appear in file explorer:
`setwd("/PHI_conf/")`

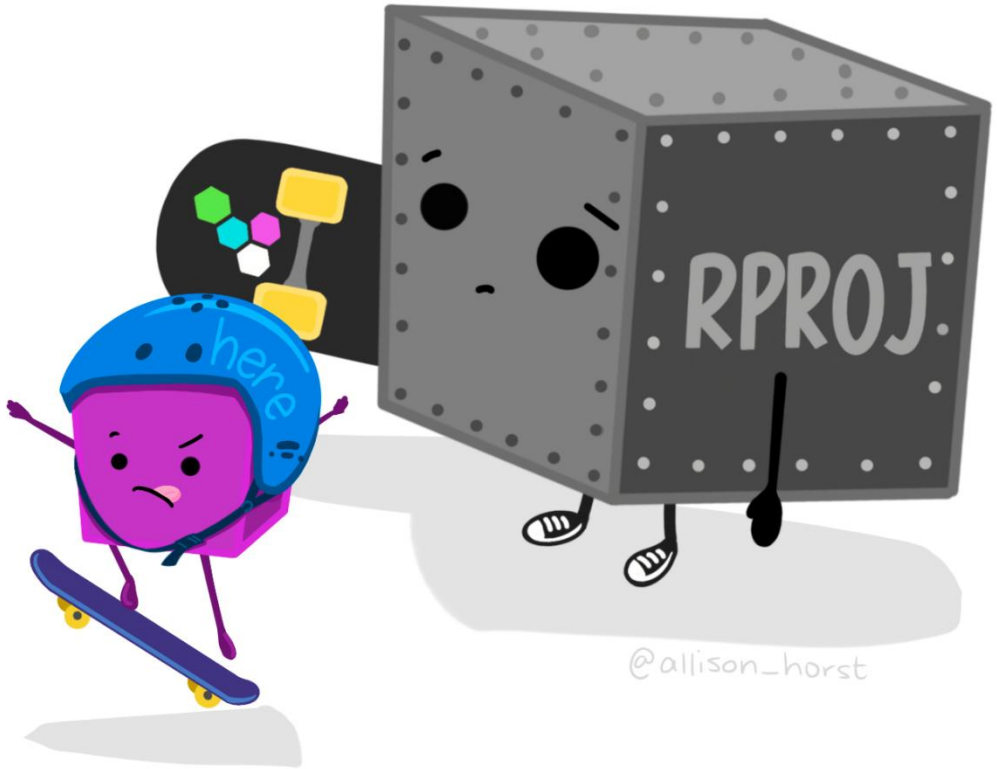
```
setwd("/home/learnr01/intro_R")
getwd()
here()
# on posit workbench:
setwd("/PHI_conf/")
```

```
> [1] "/home/learnr01/intro_R"
> [1] "/home/learnr01/intro_R"
```



Working Directory





CSV

Package: `readr`

`Read:`

1. Load package, `library(readr)`
2. `read_csv(<filepath>)`
3. Check output

Write:

1. Load package, `library(readr)`
2. `write_csv(<object>, <filepath>)`
3. Check output

```
library(readr)
borders_csv <-
     read_csv("data/Borders.csv")
View(borders_csv)
```

```
library(readr)
write_csv(borders,
          "data/borders.csv")
```

- the write functions expect a dataframe as the object.



XLSX

Package: openxlsx

Read:

1. Load package, `library(openxlsx)`
2. `read.xlsx(<filepath>)`
3. Check output

Write:

1. See next slide for process for building workbook
2. `saveWorkbook(<filepath>)`
3. Check output



XLSX

```
#xlsx
library(openxlsx)

# read in data
data_table <- read.xlsx(paste0(file_name,
".xlsx"))

##Output workbook
wb <- createWorkbook("xlsx")

## create tab
tab1 <- addWorksheet(wb, "title") # create excel
tab

## add data to tab
writeData(wb = wb, sheet = tab1,
          x = "insert text here",
          startCol = 1, startRow = 1)
```

```
## insert plots/ images
insertImage(wb, sheet = tab1,
            file = paste0(output,
"image_name.png"),
            width = 6, height = 3,
            startRow = 1, startCol = 5)

## save out
saveWorkbook(wb, <filepath>,
             overwrite = TRUE) # save
all sheets in the workbook
```



RDS

Package: `readr`

Load package, `library(readr)`

Read

- `read_rds(<filepath>)`

Write

- `write_rds(<object>, <filepath>)`

```
library(readr)


borders_RDS <-
     read_rds("data/borders.rds")


write_rds(borders,
     "data/borders.rds")
```



Parquet

Install packages: `install.packages("tidyverse")`
`install.packages("arrow")`
`install.packages("parallelly")`

Load packages: `library(tidyverse)`
`library(arrow)`

Correctly identify the number of CPUs available to the session
`n_cpus <- as.numeric(parallelly::availableCores())`

Write

- `r arrow::write_parquet(<object>, sink = "<filepath>", compression = "zstd")`

Read

- `arrow::read_parquet("<filepath>")`

```
# arrow and parquet ------------------------------------------
# Install required packages
install.packages("tidyverse")
install.packages("arrow")
install.packages("parallelly")

# Load required packages
library(tidyverse)
library(arrow)

## setup for arrow
# Correctly identify the number of CPUs available to the session
n_cpus <- as.numeric(parallelly::availableCores())

# Tell {arrow} how many CPUs it can use
arrow::set_cpu_count(n_cpus)

# save file as parquet for bonus compression
arrow::write_parquet(lookup,
                     sink = "Data/lookup.parquet",
                     compression = "zstd")

# read in compressed parquet
sc_data <- arrow::read_parquet("Data/sc_data.parquet")
```

More info on .parquet files: [PHS Data Science - Knowledge Base (public-health-scotland.github.io)]



Read SPSS

Package: `haven`

1. Load package, `library(haven)`
2. `read_sav(<filepath>)`
3. Check output

```
library(haven)
borders_spss <-
    read_sav("data/Borders.sav")
View(borders_spss)
```



Read Web

The packages/functions used will vary depending on the structure of the data. This example uses a CSV so the process to follow is the same as before.

Package: `readr`

1. Load package, `library(readr)`
2. `read_csv(<filepath>)`
3. Check output

```
library(readr)
hospital_codes <- read_csv("
    https://www.opendata.nhs.sco
t/dataset/[...].csv")
View(hospital_codes)
```



Open Data – CKAN API

Package: `ckanr`

1. Load package, `library(ckanr)`
2. Set up connection and resource ID
3. `dplyr::tbl(<src>, <res>) %>% as_tibble()`
4. Check output

```
library(ckanr)
ckan <-
     src_ckan("https://www.opendata.nhs.scot")
res_id <- ”<ID>"
resource <- dplyr::tbl(src = ckan$con, from = res_id) %>%
   as_tibble()
```



Open Data – phsopendata

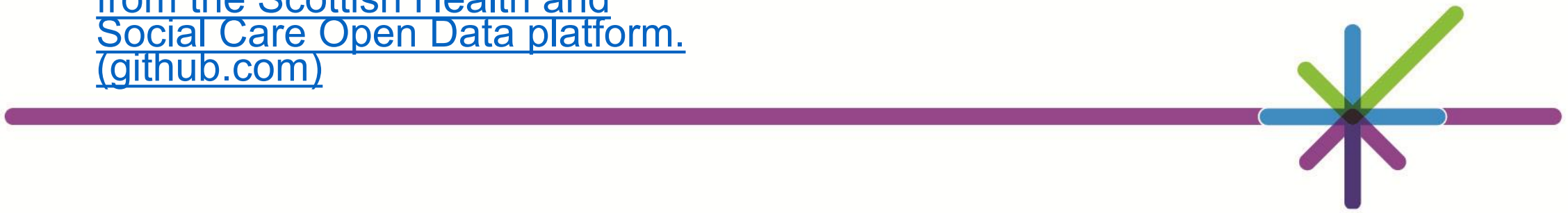
Package: `phsopendata`

1. Load package
2. Get res_id (unique identifier) for dataset **Health Board 2014 - Health Board 2019**
3. `https://www.opendata.nhs.scot/dataset/geography-codes-and-labels/resource/652ff726-e676-4a20-abda-435b98dd7bdc`
4. More info: Public-Health-Scotland/phsopendata: Functions to extract and interact with data from the Scottish Health and Social Care Open Data platform. (github.com)

```
library(phsopendata)

# Import dataset
hb_lookup <- get_resource(
  res_id = "652ff726-e676-4a20-abda-
  435b98dd7bdc")

# Only import columns for  health board
  name and 9-digit code for the same
  dataset
hb_lookup <- get_resource(
  res_id = "652ff726-e676-4a20-abda-
  435b98dd7bdc",
  col_select = c("HB", "HBName"))
```



Database (SMRA)

Package: `odbc`

1. Load package, `library(odbc)`
2. Connect:
```
smra_connection <- dbConnect(drv = odbc(),
                             dsn = "SMRA",
                             uid = .rs.askForPassword("SMRA Username:"),
                             pwd = .rs.askForPassword("SMRA Password:"))
```
3. Extract:
```
smr01 <- dbGetQuery(smra_connection,
                    paste("<sql>"))
```
4. Check output



Guidance on Filetypes

- PHS Data Science - Knowledge Base (public-health-scotland.github.io)

- PHS Data Science - Knowledge Base (public-health-scotland.github.io)



Explore

Inspect

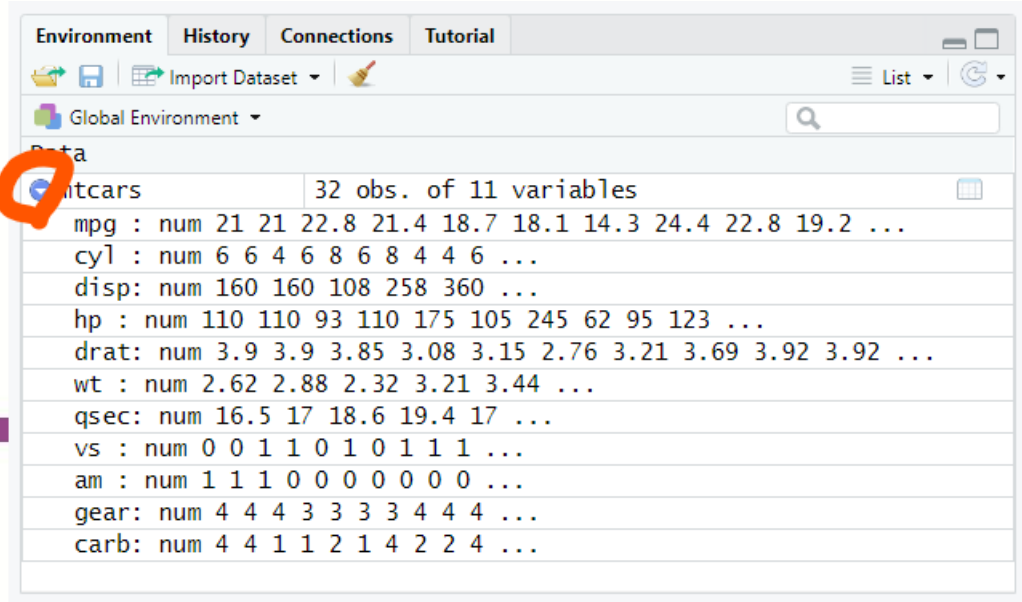
- `head()` shows the first few entries or rows of a tibble, data frame, matrix, or vector

- `str()` displays the internal structure of an R object, size, column names, data types

- `colnames()` shows the column names of a tibble or data frame

- You can also use the dropdown arrow in the environment pane (top right)

```
> # inspect
> mtcars <- mtcars
> head(mtcars)
                   mpg cyl disp  hp drat    wt  qsec vs am gear carb
Mazda RX4         21.0   6  160 110 3.90 2.620 16.46  0  1    4    4
Mazda RX4 Wag     21.0   6  160 110 3.90 2.875 17.02  0  1    4    4
Datsun 710        22.8   4  108  93 3.85 2.320 18.61  1  1    4    1
Hornet 4 Drive    21.4   6  258 110 3.08 3.215 19.44  1  0    3    1
Hornet Sportabout 18.7   8  360 175 3.15 3.440 17.02  0  0    3    2
Valiant           18.1   6  225 105 2.76 3.460 20.22  1  0    3    1
> str(mtcars)
'data.frame':   32 obs. of  11 variables:
 $ mpg : num  21 21 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 ...
 $ cyl : num  6 6 4 6 8 6 8 4 4 6 ...
 $ disp: num  160 160 108 258 360 ...
 $ hp  : num  110 110 93 110 175 105 245 62 95 123 ...
 $ drat: num  3.9 3.9 3.85 3.08 3.15 2.76 3.21 3.69 3.92 3.92 ...
 $ wt  : num  2.62 2.88 2.32 3.21 3.44 ...
 $ qsec: num  16.5 17 18.6 19.4 17 ...
 $ vs  : num  0 0 1 1 0 1 0 1 1 1 ...
 $ am  : num  1 1 1 0 0 0 0 0 0 0 ...
 $ gear: num  4 4 4 3 3 3 3 4 4 4 ...
 $ carb: num  4 4 1 1 2 1 4 2 2 4 ...
> colnames(mtcars)
 [1] "mpg"  "cyl"  "disp" "hp"   "drat" "wt"   "qsec" "vs"   "am"   "gear" "car
```



Mean/Median & Summary

- `mean()` and `median()` are passed arrays of values (usually from a data frame) to return the mean and median value

- `summary()` returns all summary statistics based on a given array (usually from a data frame)

- We access columns using `[[]]` or `$`

- `?<function>` will open the help page for a function in the bottom right window

```
mean(borders[["LengthOfStay"]])
summary(borders$LengthOfStay)
```

```
> [1] 4.297008
>   Min. 1st Qu. Median    Mean  3rd Qu.   Max
  0.000   0.000   1.000  4.297   4.000   458
```



Frequencies & Crosstabs

- Frequency: `table(<df>$<col>)`
- Crosstab: `table(<df>$<col1>, <df>$<col2>)`
- To add column and row totals, the function `addmargins()` can be used

```
addmargins(table(borders$HospitalCode, borders$Sex))
```

```
          1     2    3    Sum
A210H     1     0    0      1
B102H    56   100    0    156
B103H    50   108    0    158
...
Sum   11947 13340    2  25289
```

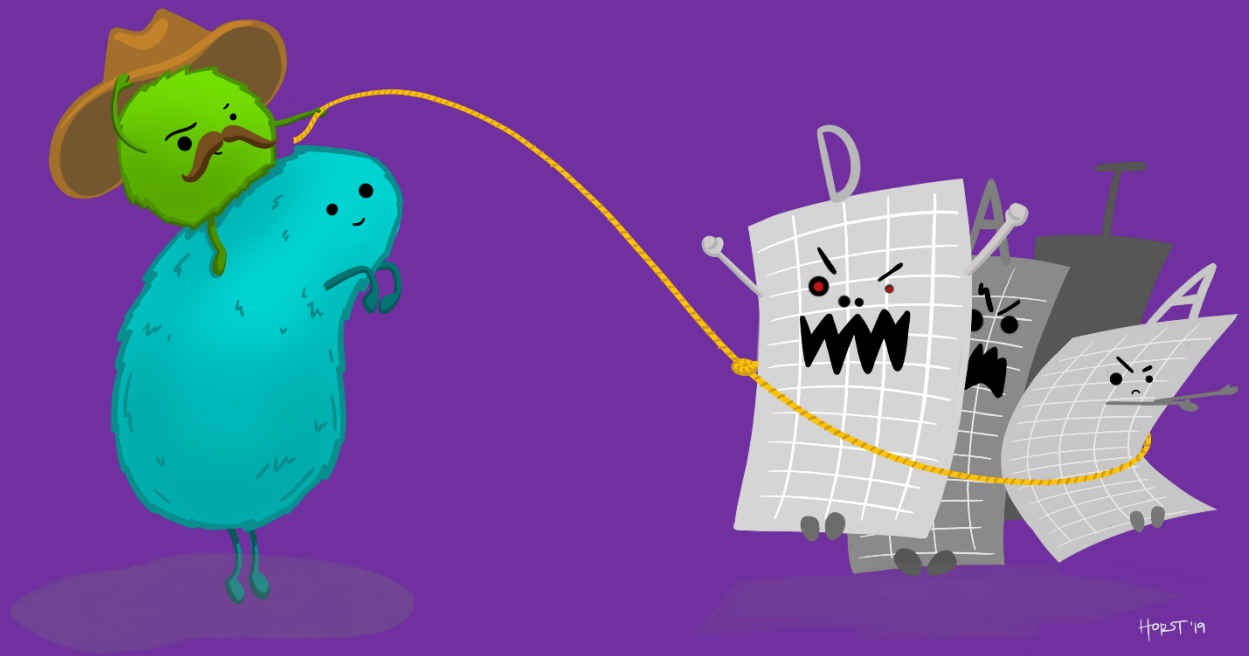


Exercise 1

1. Read in "`Borders.csv`" (giving the data frame an appropriate name)
2. What are the **mean, median, and max** values from the `LengthOfStay` variable? Can you do this in one step?
3. Produce a **frequency table** to check the `sex` variable, save this as an object with an appropriate name
4. Export the frequency table as a **csv** file. You'll need to use `as.data.frame()`

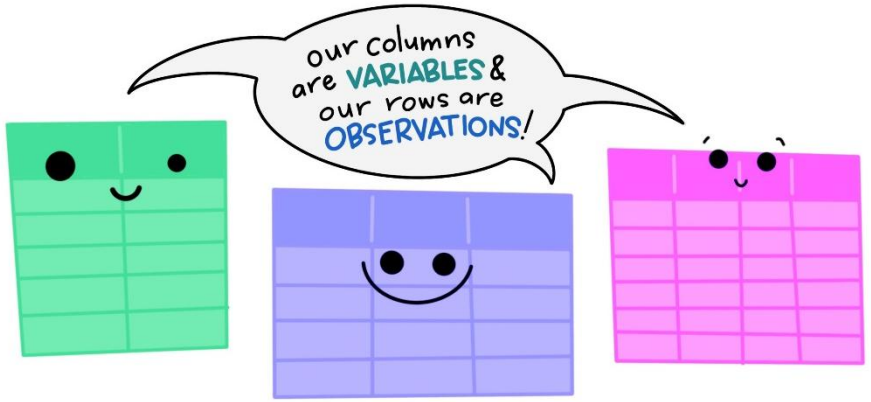


Wrangle



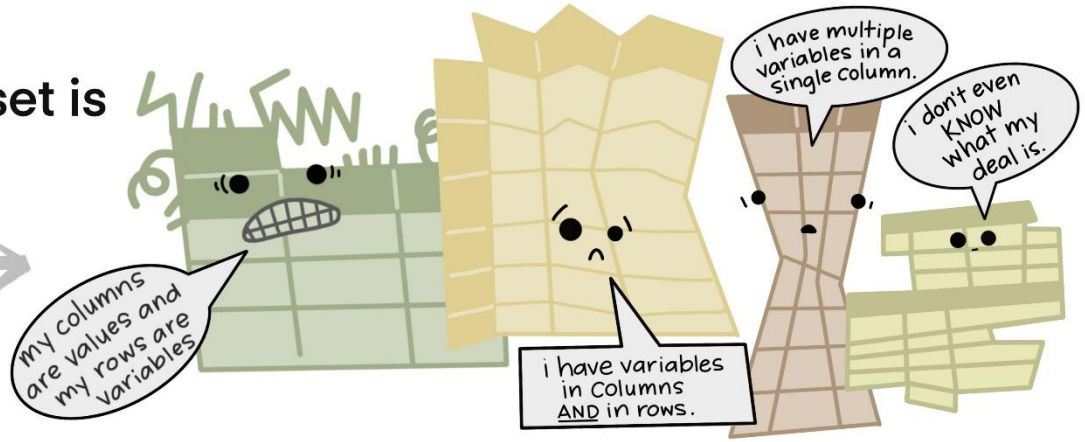
Tidy Data

The standard structure of tidy data means that "tidy datasets are all alike..."



"...but every messy dataset is messy in its own way."

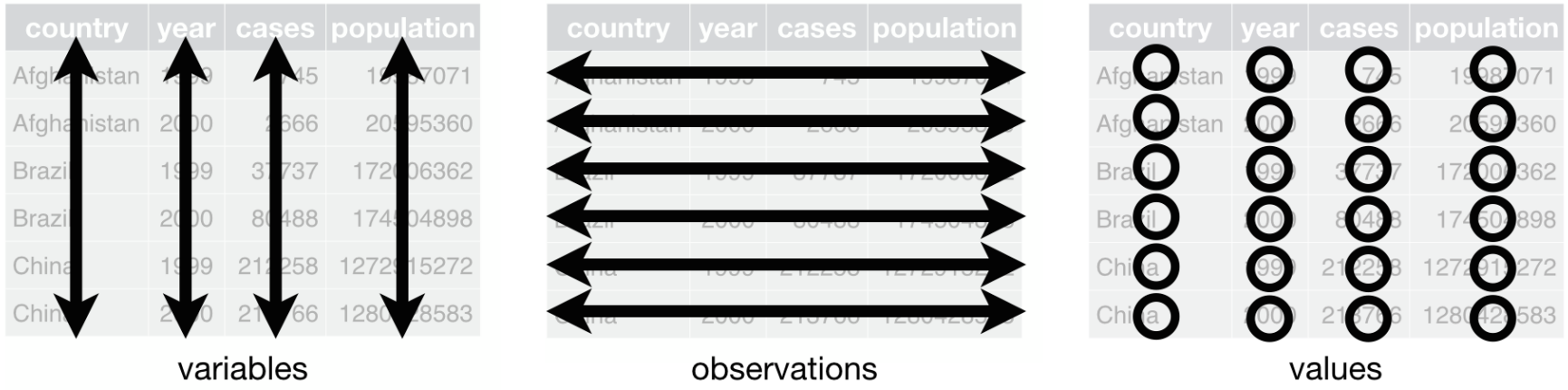
–HADLEY WICKHAM



Tidy Data

There are three interrelated rules that make a dataset tidy:

1. Each variable is a column; each column is a variable.
2. Each observation is a row; each row is an observation.
3. Each value is a cell; each cell is a single value.



From R for data science, Hadley Wickham: [R for Data Science (2e) (hadley.nz)]

Tidyverse

is a suite of packages for data exploration, manipulation, and visualisation; it's best practice to utilise these where possible.



- functions have a consistent format, i.e. `function(data, task)`

- gives us the package `dplyr`

dplyr

is a grammar of data manipulation, providing a set of "*verbs*" (functions) to help solve most data manipulation challenges

```
library(dplyr)
```

- `filter()`
- `mutate()`
- `arrange()`
- `select()`
- `group_by()`
- `summarise()`
- `count()`
- `rename()`
- `recode()`



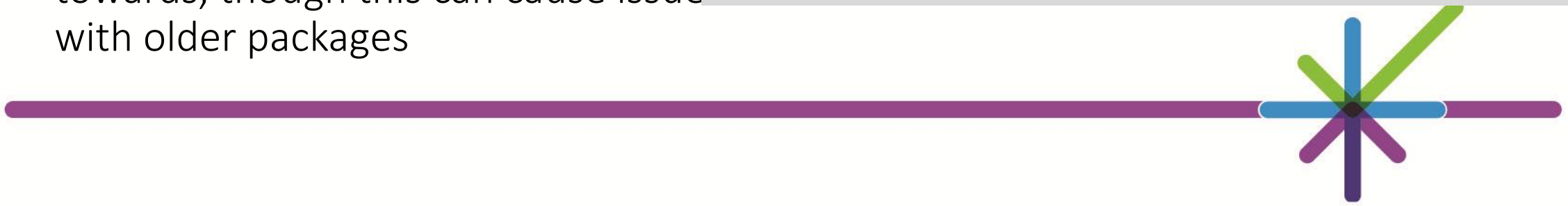
More info: [A Grammar of Data Manipulation • dplyr (tidyverse.org)]

Pipe Operator

- `%>%` is used to link functions together, passing the previous to the next
- Using the pipe operator makes R code more readable and prevents extensive brackets building up with multiple function calls
- Readable as "and then"
- Shortcut: (`ctrl` + `shift` + `m`)
- Newer code might write this as `|>` which is what tidyverse is moving towards, though this can cause issue with older packages

```
arrange(filter(rename(borders,
date_of_birth = Dateofbirth),
HospitalCode == "B102H"),
date_of_birth)
```

```
borders %>%
    rename(date_of_birth = Dateofbirth) %>%
    filter(HospitalCode == "B102H") %>%
    arrange(date_of_birth)
```



Pipe Operator

```
borders %>%
    filter( HospitalCode == "B102H") %>%
    arrange( Dateofbirth)%>%
    rename( date_of_birth =
    Dateofbirth)
```



Filter

`filter(<data>, <logical expression>)`

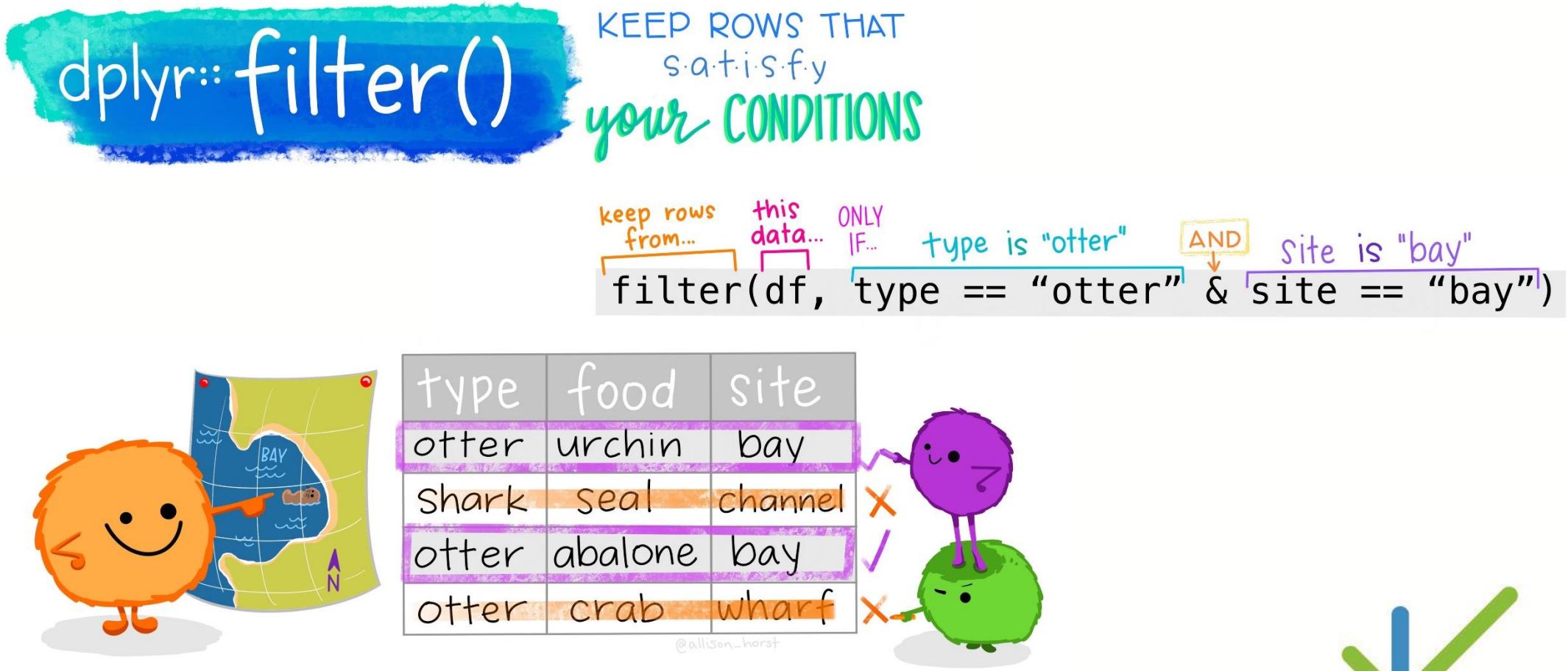
- picks cases based on their values

```
# all cases with E12 specialty
borders %>%
     filter(Specialty == "E12")

# B120H cases more than 10 days
borders %>%
     filter(HospitalCode ==
                "B120H" &
            LengthOfStay > 10)
```



Filter


dplyr::filter()
KEEP ROWS THAT
s·a·t·i·s·f·y
your CONDITIONS
keep rows from...
this data...
ONLY IF...
type is "otter"
AND
site is "bay"
filter(df, type == “otter” & site == “bay”)
BAY
N
type food site
otter urchin bay
shark seal channel
otter abalone bay
otter crab wharf
@allison_horst

Mutate

```
mutate(<data>, <new_col> =
<expression>)
```

- adds new variables that are functions of existing variables

```
# length of stay divided by 2
borders %>%
    mutate(los_div2 =
        LengthOfStay / 2)
```



Mutate


dplyr::mutate
add Column(s),
keep existing.
A
B
C
1
5
10
0
2
-3
4
9
6
3
2
14
9
8
THE PLAN:
A
B
C
Horst '18

Arrange

`arrange(<data>, <variables>)`

- changes the ordering of rows
- `desc()` to sort in descending order

```
# sort by Hospital Code
borders %>%
    arrange(HospitalCode)
```



Select

`select(<data>, <expression>)`

- picks variables based on their names
- prepend "–" to a variable to remove

```
# remove Postcode
borders %>%
    select(-Postcode)
```



Exercise 2

1. Read in "`Borders.csv`" (giving the data frame an appropriate name)
2. Which patients had a `LengthOfStay` of between 2 and 10 days?
3. Which of these patients were under `Specialty` E12 or C8?
4. Remove all columns other than `URI`, `Specialty`, and `LengthOfStay`
5. Complete all the above using pipes and write this to a CSV ordered by `LengthOfStay`



Group By

```
group_by(<data>,
<col_name>, <col_name>)
```

- groups variables to perform operations

- This doesn't visibly affect the data, but we can see the output shows the grouping. We can then perform other operations on the groups.

```
# sort by Hospital Code
borders %>%
    group_by(HospitalCode)
```

```
> ...
# Groups: HospitalCode [48]
...
```



Summarise

```
summarise(<data>, <name> = <expression>)
```

- reduced multiple values down to a single summary for each combination of grouped columns

```
# avg length of stay by hospital
borders %>%
    group_by(HospitalCode) %>%
    summarise(mean_los =
        mean(LengthOfStay))
```



Count

`count(<data>, <variables>)`

- useful for running frequencies, this calls group_by() and produces counts for a specified column

- sort by descending order using `sort = TRUE` as an argument

```
# counts of specialty
borders %>%
     count(Specialty, sort = TRUE)
```



Exercise 3

1. Read in "`Borders.csv`" (giving the data frame an appropriate name)
2. What is the earliest admission date by specialty?
3. What is the latest discharge date by specialty?
4. What are the number of admissions per hospital, per specialty?



Rename

```
rename(<data>, <new_name> = <existing_name>)
```

- renaming specific columns in a data frame

```
# rename Date of Birth column
borders %>%
    rename(date_of_birth = Dateofbirth)
```



Recode

```
mutate(<col> = recode(<col>,
<existing_code> =
<new_code>))
```

- for changing values within a column
- works best when used with `mutate()`

```
# change hospital code
borders %>%
    mutate(HospitalCode =
        recode(HospitalCode,
            "B120V" = "B120H"))
```



Exercise 4

1. Select the `URI`, `Specialty`, and `Dateofbirth` columns from the borders data and save to a new data frame.
2. Arrange this new data in ascending order by `Specialty` and check the results.
3. Extract the records with a missing `Dateofbirth` (hint: `?filter`)
4. Finally, recode `Specialty` "A1" to be "General Medicine"



Joining Tables

```
<type>_join(<data1>,
<data2>, by =
<common_variable>)
```

- for merging data by matching together using common variable(s)

- More info on types of joins: R for Data Science (2e) - 19 Joins (hadley.nz)

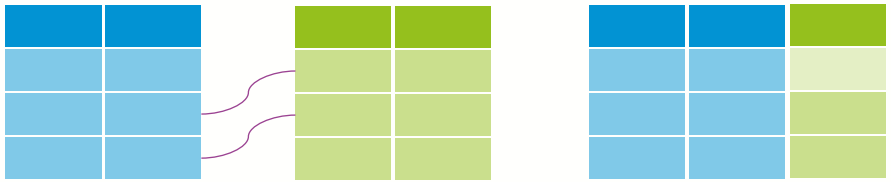
```
# merge baby data
baby5 <- read_csv("data/Baby5.csv")
baby6 <- read_csv("data/Baby6.csv")
baby_joined <- left_join(baby5, baby6,
                         by = c("FAMILYID", "DOB"))
baby_full <- full_join(baby5, baby6,
                       by = c("FAMILYID", "DOB"))
```



Join Types

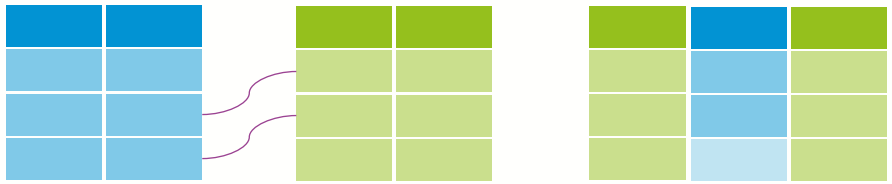
`left_join()`

`all rows from the 'left', any matches from the 'right'`



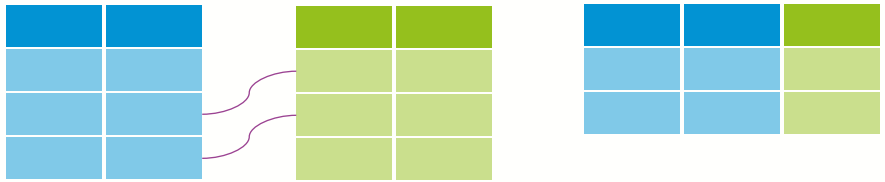
`right_join()`

`all rows from the 'right', any matches from the 'left'`



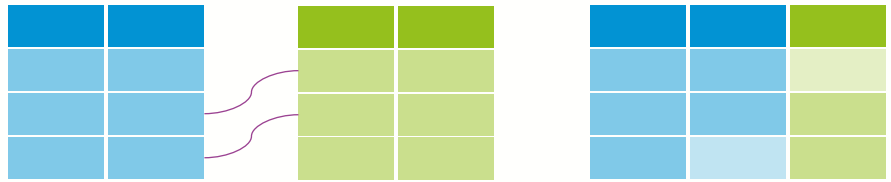
`inner_join()`

`rows of matched fields from data sets`



`full_join()`

`all rows, na for non-matched fields`





General Skills

Debugging

1. **Review warnings/errors** – these can appear cryptic but use Google and some will become familiar. Checking the functions could help – `?<function>`

2. **Narrow the problem** – step through the code one line at a time, isolating the issue.

3. **Google/Stack Overflow** – this can be specific to the bug or more general to the problem you're trying to solve.

4. Pair up – sometimes a fresh pair of eyes makes the difference. Post a message on the PHS Data and Intelligence Forum | Posit Technical Queries | Microsoft Teams or PHS Data and Intelligence Forum | Beginners Channel | Microsoft Teams



Debugging



1.
I got this.



2.
Huh. Really thought that was it.



3.
(...)



4.
Fine. Restarting



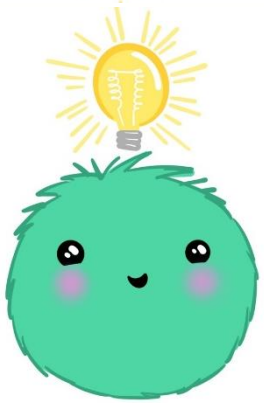
5.
OH WTF.



6.
Zombie meltdown



7.



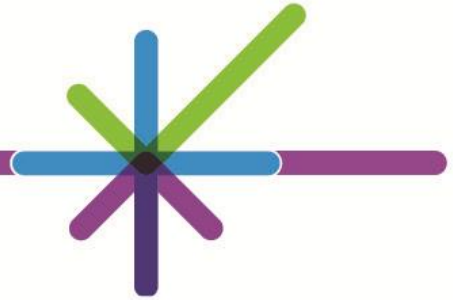
8.
A NEW HOPE!

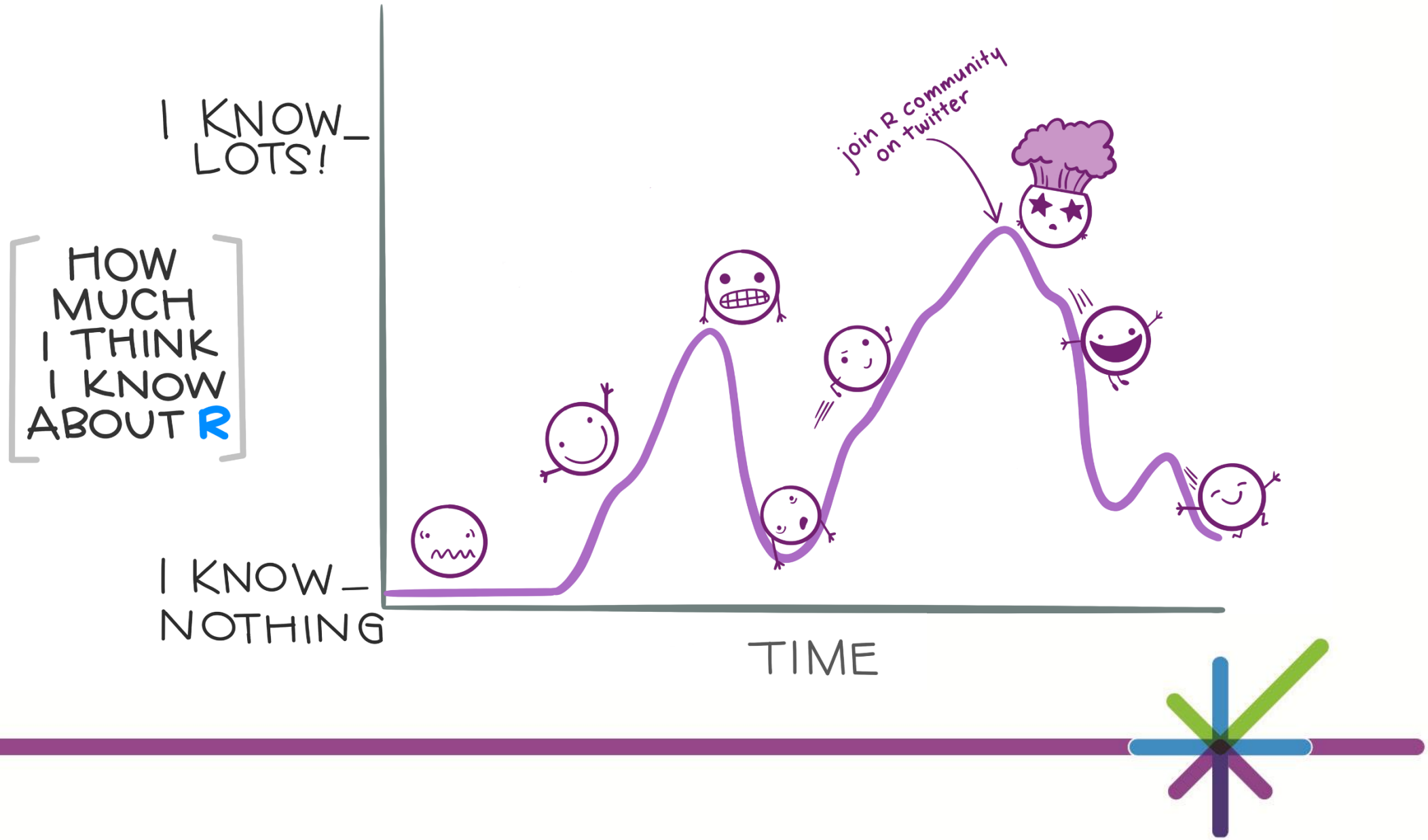


9.
[insert awesome theme song]



10.
I ♥ CODING!




I KNOW LOTS!
HOW MUCH I THINK I KNOW ABOUT R
I KNOW NOTHING
TIME
join R community on twitter

Continuous Learning

- Data Science Knowledge Base **is** the place for all content related to Data Science learning:
  - **Review, Follow, and Contribute to Guidance** – guidance is for sharing best practice, maintaining security, and improving efficiency.
  - **Expand your skills** – take another course to build your R skills or on related technologies (e.g. Git).
  - **Keep up to date** – our infrastructure is improving, we support knowledge sharing events, and so much more!



Review

Project

scotland.shinyapps.io/phs-rtraining-intro/

- Day 2 Project – Handwashing

Feel free to follow along for the project on the app or build a script on Posit Workbench



Next Steps

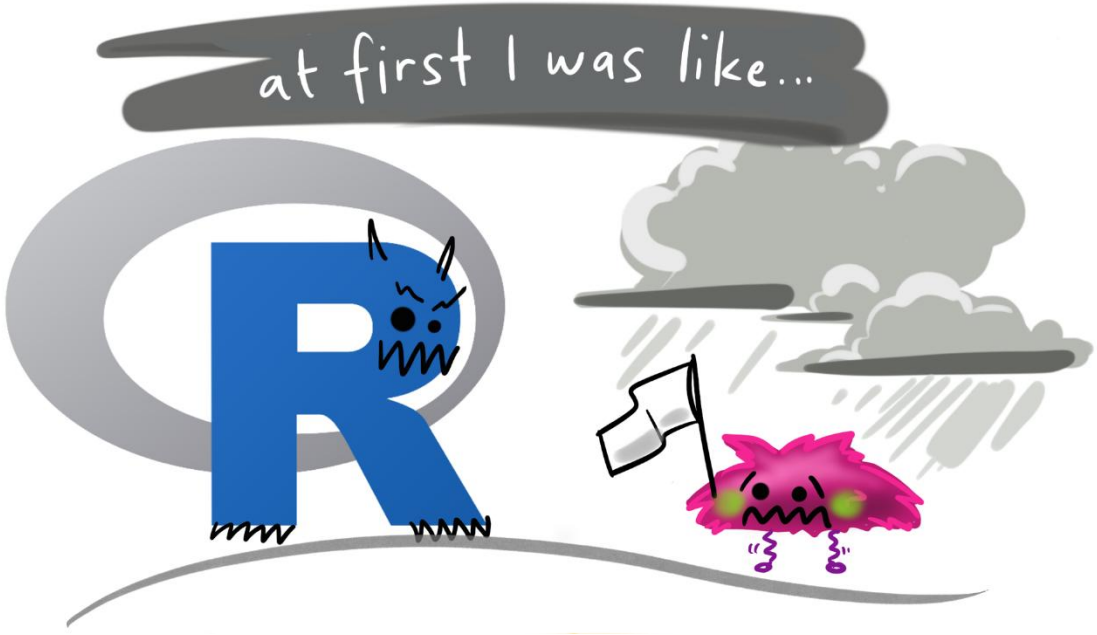
- Homework project & day 3
- Embed your new knowledge and skills!
- Expand your knowledge and skills with related technologies (e.g. git)
- Take R Further - look at other training opportunities (`phsmethods`)



Getting Help

- Vignettes (Help) / `` `?<function>` ``
- Google / Stack Overflow
  - tag queries "[r] & [tidyverse]“
- PHS Data Science - Knowledge Base (public-health-scotland.github.io)
- PHS Data and Intelligence Forum:
  - Technical Queries
  - PHS Data and Intelligence Forum | Beginners Channel | Microsoft Teams







All artwork in these slides is by @allison_horst



Feedback

- Spend 5 Minutes making some notes
  - (somewhere you won't lose them!)
  - There will be a full **feedback form** at the **end** of **session 3**
  - Things to consider:
    - How was the **pre-course checklist** and **session**?
    - How do you feel about the **content** of the course?
    - How do you feel about the **way the course was delivered**?
    - Has your level of **confidence** and **knowledge changed**?
    - What did you find **useful**? What did you **not** find useful?
    - What did you **enjoy**? What did you **not** enjoy?
    - Did you **learn** what you **needed and wanted to** learn?
    - What could be **improved** for future trainees?



Feedback Form

- Form: https://forms.office.com/e/aDw8r4XmmC
  - (will **not** record your name!)

